

KV Cache Memory Management

Deep Dive into SGLang — Chapter 7

Yin Yang

Foundation Books Project

Where this chapter sits

This is the chapter that turns “the KV cache is full” into a precise **ownership** problem.

- Chapter 1 said one cached position costs ≈ 84 KiB; Chapter 5 said memory is the binding constraint.
- Here we ask the harder question: *when is a physical slot actually safe to reuse?*
- The answer is a protocol, not a counter — and it is the spine of prefix reuse, chunked prefill, overlap, and speculation.

Goal: prove a slot is free only after no request row, cache record, or host copy can still name it.

The ownership problem

Three address spaces

Allocation and pressure

Release, movement, variants

The ownership problem

Allocation is an ownership protocol, not a free count

A KV page can outlive the forward call that wrote it.

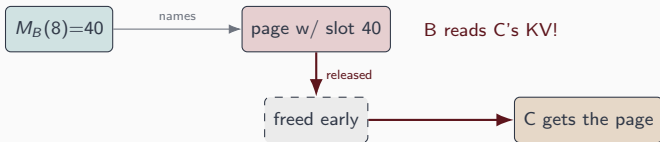
- A live request row may still read it.
- A prefix-cache entry may protect it for reuse.
- A host tier may hold a copy; a speculative path may hold it provisionally.

KV ownership contract

$M_r(j) = v$ is valid only while r or protected reusable state owns the page containing v . A page joins the free pool only after *no* live row or protected record can read it.

Page counts say how much memory exists; ownership says whether reuse is safe.

The failure it prevents: a stale row



If B's row still holds $M_B(8)=40$ but the page was freed and reassigned to C, B's next attention read names **C's history**.

It surfaces as an attention bug; the root cause is a memory-ownership bug. Correct release clears the row *before* the page is reusable.

Three address spaces

Keep four things separate

Object	What it names	Common mistake
Logical position j	token offset in one request	treating it as an address
Request row M_r	map: positions $\rightarrow$ slots	assuming contiguous memory
KV slot v	one physical K/V location	adjacent tokens $\rightarrow$ adjacent slots
Allocator page	unit of ownership / release	counting slack as free

The logical row is contiguous; the physical slots need not be. A row can read [20:24] then jump to [40:44] as one logical sequence.

Page geometry: where token math meets physical memory

Logical length ℓ owns $\text{pages}_P(\ell) = \lceil \ell/P \rceil$ pages. The last page is owned even when partly empty — that is **page slack**.



Page slack is owned memory, not globally free memory. Only the owner may fill it.

Allocation and pressure

Prefill vs. decode: the same page rule, two shapes

Prefill append ($p \rightarrow n$):

$$\Delta_{\text{pages}} = \lceil \frac{n}{P} \rceil - \lceil \frac{p}{P} \rceil$$

suffix of 8 tokens may need only 2 pages (fills slack first).

Decode step ($\ell \rightarrow \ell+1$):

$$\lceil \frac{\ell+1}{P} \rceil - \lceil \frac{\ell}{P} \rceil \in \{0, 1\}$$

one token: free mid-page, a whole page at a boundary.

Logical tokens and pages move in opposite directions. Counting tokens alone over-admits work that does not physically fit.

Memory pressure: the capacity ledger

Pressure is when near-term demand exceeds free *plus* evictable slots:

$$\underbrace{A_t}_{\text{free}} + \underbrace{|E_t|}_{\text{evictable}} \geq \text{page-aligned reserve.}$$

- **Protected** resident KV occupies memory but is *not* on the usable side.
- The reserve includes near-future *decode* work, not just this step's suffix.
- Fallback ladder: **evict** prefix cache → **retract** decode rows → **fail closed**.

Each ownership or residency change is followed by a budget recheck — the order is a safety argument.

From concept to code: the feasibility test

The whole pressure check lives in one method.

```
from python/sglang/srt/managers/schedule_batch.py

def check_decode_mem(self, selected_indices=None):
    num_tokens = self.new_tokens_required_next_decode(selected_indices)
    evict_from_tree_cache(self.tree_cache, num_tokens)
    return self.token_to_kv_pool_allocator.available_size() >= num_tokens
```

Read it as the ledger: compute the page-aligned next-decode reserve, ask the prefix cache to evict what it safely can, then compare demand against `available_size()`. That is $A_t + |E_t| \geq \text{reserve}$.

Release, movement, variants

Release, retraction, and host movement: one invariant

Release invariant

A freed page set F must satisfy $F \cap \text{protected}(S') = \emptyset$: no protected logical position may read a freed page.

- **Release:** clear row, drop protection, *then* free — reclaim overallocated $[c_r, a_r)$ separately.
- **Retraction:** free now, but the request stays schedulable; old slots must vanish from its row.
- **Host movement:** physical slots may change; logical positions may *not*. The layer counter gates each layer read.

Same contract, different live set.

Model variants change bytes, not the boundary

Family	Allocator contract	Per-token bytes (2B elt)
MHA/GQA	K/V rows: $\text{heads} \times \text{head-dim} \times \text{layers}$	Llama-3.1-8B: ≈ 128 KiB
MLA	latent rows + RoPE dim per layer	DeepSeek-V3: ≈ 68.6 KiB
SWA	slots evictable once outside window <i>and</i> unprotected	—
Mamba	request-owned recurrent state	not disposable slack

The representation changes the bytes, the auxiliary buffers, and the eviction test — but the allocator still needs owned locations and capacity in its own units.

What to carry forward

1. KV management is an **ownership protocol**: free only at the *zero-claim* state.
2. Feasibility is **page-aligned**; one token can cost 0 or a whole page.
3. **Page slack** is owned; **allocated** can exceed **committed** state.
4. Admission, eviction, release, retraction, host movement are the **same check** under a changing live set.
5. Every concept has a home in `mem_cache/` — pools, allocator, host pools, and the transfer kernels.

Next: Chapter 8 builds the structure that finds, protects, and evicts those reusable prefix slots — RadixAttention and prefix reuse.